Laboratorio 4 — Actas municipales en un grafo

TI-4601 Bases de Datos Avanzadas · Grupo 3 · Fabricio Mena · Alisson Redondo · Christopher Rodriguez

Repositorio: `https://github.com/fabrimena/Lab-4-Bases-de-Datos-Avanzadas` (notebook en `labs/lab4-neo4j/Lab4.ipynb`)

Entorno. Laptop AMD Ryzen 7 5700U, 16 GB de RAM (4 CPU y 9,7 GiB para WSL 2), Windows 11 Home con Ubuntu 24.04 en WSL 2 y Docker Desktop 29.5.2. Neo4j 5.26.32 Community, driver de Python 5.28.8 y LlamaIndex 0.14.25. Las salidas pegadas son de la ejecución final del 11 de noviembre, con el grafo vacío al empezar y el notebook completo con `make lab4-ejecutar`; es la que queda guardada en el repositorio.

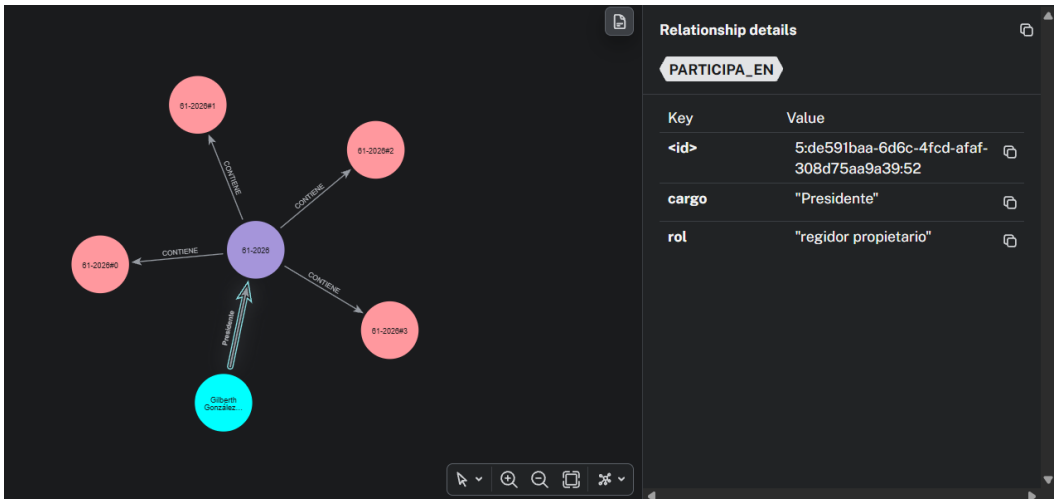
1. Delimitación (pregunta 1)

El índice de actas municipales de Belén publica 4234 documentos. Para el laboratorio se seleccionaron cuatro actas: 58-2026, 59-2026, 60-2026 y 61-2026. De cada PDF se extrajo el texto de sus primeras dos páginas, con el cual se construyó el grafo en Neo4j.

```
El índice publica 4234 documentos. El corpus usa 4.
OK: 58-2026 tiene texto extraíble en sus primeras 2 páginas
OK: 59-2026 tiene texto extraíble en sus primeras 2 páginas
OK: 60-2026 tiene texto extraíble en sus primeras 2 páginas
OK: 61-2026 tiene texto extraíble en sus primeras 2 páginas
```

Si un PDF fuera únicamente una imagen escaneada, no sería posible extraer directamente su contenido como texto. Se aplicaría reconocimiento óptico de caracteres (OCR), seguido de una revisión y limpieza del texto obtenido, antes de incorporarlo al proceso de fragmentación y carga en Neo4j.

2. El grafo en el navegador y el cargo (pregunta 2)



Paso 6.3 en el navegador de Neo4j: el camino de Gilberth a la 61-2026 y sus cuatro fragmentos, con la relación `PARTICIPA_EN` seleccionada.

En la captura, el cargo y el rol aparecen en el panel de la relación `PARTICIPA_EN`; el nodo `Persona` solo guarda el nombre. `Persona` representa a la persona y `PARTICIPA_EN` representa su presencia en una sesión. El cargo describe lo segundo: es lo que esa persona fue *en esa acta*. La misma persona aparece en varias actas; en nuestro grafo, 17 de las 18 personas están en las cuatro y una en tres.

Si el cargo fuera una propiedad de `Persona`, tendría un solo valor para todas sus sesiones. Cuando cambie, por ejemplo porque se elige otra presidencia o porque en una sesión preside la vicepresidenta, el `SET` de la carga sobrescribiría el valor y el grafo diría que la persona siempre tuvo el último cargo cargado. Se pierde la historia y ya no se puede responder «¿quién presidió la 58-2026?». En la relación, cada acta guarda su propio valor, y la consulta pregunta por la relación y no por la persona. Es lo que comprueba la verificación del TODO 1:

```
OK: el cargo vive en la relación: presidencia de 61-2026 = ['Gilberth González González']
```

En estas cuatro actas nadie cambió de cargo (lo revisamos con una consulta aparte): Gilberth González tiene cuatro relaciones con `cargo = 'Presidente'`, una por acta. Con más actas sí habría cambios, y el modelo ya los admite sin tocar el esquema. Lo mismo vale para `rol`: alguien puede ser suplente en un periodo y propietario en otro.

3. Consultas de uno y dos saltos (pregunta 3)

TODO 3: un salto. Se recorre la relación `CONTIENE` entre cada acta y sus fragmentos. `count(f)` permite agruparlos por acta sin utilizar `GROUP BY`. La consulta a la izquierda y el SQL equivalente a la derecha:

```
MATCH (a:Acta)-[:CONTIENE]->(f:Fragmento)
RETURN a.id AS acta, a.tipo AS tipo,
       a.fecha AS fecha, count(f) AS fragmentos
ORDER BY acta ASC
```

```
SELECT a.id AS acta, a.tipo, a.fecha,
       COUNT(*) AS fragmentos
FROM acta a
JOIN fragmento f ON f.acta_id = a.id
GROUP BY a.id, a.tipo, a.fecha
ORDER BY a.id;
```

Salida del notebook:

```
   {'acta': '58-2026', 'tipo': 'Ordinaria', 'fecha': '2026-09-29', 'fragmentos': 4}
   {'acta': '59-2026', 'tipo': 'Extraordinaria', 'fecha': '2026-10-01', 'fragmentos': 3}
   {'acta': '60-2026', 'tipo': 'Ordinaria', 'fecha': '2026-10-06', 'fragmentos': 4}
   {'acta': '61-2026', 'tipo': 'Ordinaria', 'fecha': '2026-10-13', 'fragmentos': 4}
OK: una fila por acta, con las columnas acta, tipo, fecha y fragmentos
```

TODO 4: dos saltos. Se recorre el camino `Persona -> Acta -> Fragmento`. La consulta filtra por participante y por palabra sin distinguir mayúsculas de minúsculas.

```
MATCH (p:Persona)-[:PARTICIPA_EN]->(a:Acta)
      -[:CONTIENE]->(f:Fragmento)
WHERE p.nombre = $nombre
  AND toLower(f.texto) CONTAINS toLower($palabra)
RETURN a.id AS acta, f.id AS fragmento
ORDER BY acta ASC, f.indice ASC
```

```
SELECT a.id AS acta, f.id AS fragmento
FROM persona p
JOIN participacion r ON r.persona_id = p.id
JOIN acta a ON a.id = r.acta_id
JOIN fragmento f ON f.acta_id = a.id
WHERE p.nombre = :nombre
  AND LOWER(f.texto) LIKE
      '%' || LOWER(:palabra) || '%'
ORDER BY a.id, f.indice;
```

Salida del notebook:

```
OK: el corpus tiene al menos un caso con resultados

   Ana Betty Valenciano Moscoso + «pésame»: ['61-2026#1', '61-2026#2']
OK: dos saltos correctos para Ana Betty Valenciano Moscoso y «pésame»
   Zeneida Chaves Fernández + «aprobación»: ['58-2026#0', '58-2026#1', '60-2026#0', '60-2026#1', '61-2026#0', '61-2026#1']
OK: dos saltos correctos para Zeneida Chaves Fernández y «aprobación»
```

Comparación. El TODO 3 requiere un `JOIN` en SQL para relacionar actas y fragmentos. El TODO 4 necesita tres `JOIN` porque la participación se representa mediante una tabla intermedia. Un tercer salto desde los fragmentos hacia otra entidad requeriría al menos un `JOIN` adicional, según el esquema relacional. En Cypher basta con extender el patrón mediante otra relación, lo que facilita expresar recorridos de varios saltos en el grafo.

4. `CREATE` contra `MERGE` (pregunta 4)

Estas son nuestras consultas de los TODO 1 y 2. Siguen el ejemplo de las actas: `MATCH` del acta, `MERGE` por la clave única y `SET` del resto.

```
// TODO 1
MATCH (a:Acta {id: $id})
MERGE (p:Persona {nombre: $nombre})
MERGE (p)-[r:PARTICIPA_EN]->(a)
SET r.rol = $rol, r.cargo = $cargo
```

```
// TODO 2
MATCH (a:Acta {id: $acta})
MERGE (f:Fragmento {id: $id})
SET f.indice = $indice, f.texto = $texto,
    f.embedding = $embedding
MERGE (a)-[:CONTIENE]->(f)
```

Salida del notebook. Cada verificación carga dos veces y compara los conteos:

```
OK: cargar dos veces no duplica nada: (18, 71) → (18, 71)
OK: (personas, relaciones) = (18, 71); se esperaba (18, 71)
OK: cargar dos veces no duplica fragmentos ni relaciones
OK: conteos {'fragmentos': 15, 'contiene': 15, 'sigue': 11};
    se esperaba {'fragmentos': 15, 'contiene': 15, 'sigue': 11}
OK: todos los fragmentos tienen su vector
```

Para ver qué pasa con `CREATE`, corrimos una copia del notebook hasta el paso 5.3 con dos variantes del TODO 1. El notebook del repositorio no se tocó.

a) `CREATE` para la persona y para la relación.

```
ConstraintError: {code: Neo.ClientError.Schema.ConstraintValidationFailed} {message: Node(0) already
exists with label `Persona` and property `nombre` = 'Gilberth González González'}
VERIFICACIÓN FALLÓ: la carga intentó crear de nuevo un nodo que ya existía. Use MERGE (no CREATE)
para la persona y para la relación.
conteo (personas, relaciones): (18, 18)
```

No llega a la segunda carga: falla en la primera, al pasar a la segunda acta. La 58-2026 crea sus 18 personas, pero el primer participante de la 59-2026 es Gilberth, que ya existe, y la restricción `persona_nombre` rechaza el segundo nodo. Como cada `correr` es su propia transacción, lo anterior queda guardado y el grafo queda a medias: solo la 58-2026 tiene participantes. Sin la restricción no habría error, pero quedarían 71 nodos `Persona` en lugar de 18, uno por participación, y las cuatro sesiones de Gilberth no estarían unidas por el mismo nodo.

b) `MERGE` para la persona y `CREATE` para la relación.

```
VERIFICACIÓN FALLÓ: cargar dos veces no duplica nada: (18, 71) → (18, 142)
```

Aquí Neo4j no da ningún error, porque la restricción es sobre `Persona.nombre` y no hay ninguna sobre relaciones. La primera carga deja bien los conteos, pero la segunda duplica las 71 relaciones: Gilberth queda con dos `PARTICIPA_EN` hacia cada acta, las dos con `cargo = 'Presidente'`, y cualquier conteo de participaciones sale doble. Es el caso más peligroso porque falla en silencio; solo lo detecta la verificación que carga dos veces.

Con `MERGE` en los dos lugares la carga es idempotente: la segunda vez encuentra los mismos nodos y relaciones y solo vuelve a aplicar el `SET`. La restricción protege los nodos, pero las relaciones solo las protege el `MERGE`.

5. Solo vector contra vector + acta (pregunta 5)

Nuestra consulta del TODO 5 va después de `YIELD node, score`. Sube por `CONTIENE` hasta el acta, filtra por `$ids` y recorre `SIGUE` y `PARTICIPA_EN` con `OPTIONAL MATCH` para que la fila no se pierda si no hay fragmento siguiente:

```
MATCH (a:Acta)-[:CONTIENE]->(node)
WHERE size($ids) = 0 OR a.id IN $ids
OPTIONAL MATCH (node)-[:SIGUE]->(siguiente:Fragmento)
OPTIONAL MATCH (p:Persona)-[r:PARTICIPA_EN]->(a)
WITH node, score, a, siguiente, collect({nombre: p.nombre, rol: r.rol, cargo: r.cargo}) AS participantes
RETURN node.id AS fragmento, node.texto AS texto, siguiente.texto AS siguiente, a.id AS acta,
       a.tipo AS tipo, a.fecha AS fecha, a.url AS url, score, participantes
ORDER BY score DESC LIMIT $k
```

Salida del paso 7 para «¿Quién preside la sesión 61-2026?» (las siete verificaciones dan `OK`):

```
Solo vector:    [('60-2026', 0.819), ('58-2026', 0.814), ('61-2026', 0.81)]
Vector + acta:  [('61-2026', 0.81), ('61-2026', 0.646), ('61-2026', 0.636)]
```

Solo vector se equivoca de acta. Sin el filtro, el primer fragmento es de la 60-2026 y el segundo de la 58-2026; el acta que pide la pregunta queda tercera. Los tres son el fragmento `#0` de su acta, es decir, el encabezado: «Acta de la Sesión Ordinaria N° … celebrada por el Concejo Municipal de Belén… » seguido de la misma lista de participantes. Entre los tres solo cambian el número y la fecha, y la diferencia de score entre el primero y el tercero es de 0,009. La búsqueda vectorial mide qué tan *parecido* es el texto, y los tres encabezados se parecen igual a la pregunta. Con el filtro, el índice devuelve los 15 candidatos y el camino `CONTIENE` descarta los que no son de la 61-2026: los tres resultados son de esa acta, y el primero es su encabezado, que nombra al presidente. El acta correcta sale de una relación del grafo, no de una similitud.

El papel de los 128 tokens. Salida del paso 8.2:

```
Tokens del codificador: ['_¿', 'Quién', '_pre', 'side', '_la', '_sesión', '_61', '-20', '26', '?']
El codificador lee como máximo 128 tokens por texto.
Los fragmentos tienen entre 91 y 375 tokens: 12 de 15 superan el límite.
   fragmento   coseno   (1 + coseno) / 2
   60-2026#0   0.638    0.819
   58-2026#0   0.627    0.814
   61-2026#0   0.620    0.810
```

12 de los 15 fragmentos superan el límite, y de ellos el codificador solo ve los primeros 128 tokens. En los encabezados, esos 128 tokens son casi todos el mismo texto fijo y la lista de participantes, que es igual en las cuatro actas. Lo único que distingue a la 61-2026 es un token (`_61` frente a `_60` o `_58`), y en un vector de 384 dimensiones que resume todo el texto ese token pesa muy poco. Lo que viene después del recorte (los acuerdos, las notas de pésame) no influye en el vector, así que tampoco ayuda a separar las actas. La tabla confirma además la escala: el score de Neo4j es $(1 + \cos)/2$; por ejemplo, $(1 + 0{,}638)/2 = 0{,}819$, y el orden es el mismo que el del cálculo en Python.

6. Lectura crítica del modelo (pregunta 6)

Salida del paso 9 (Qwen2.5-0.5B-Instruct, decodificación voraz):

```
Pregunta: Según el acta 61-2026, ¿quién preside la sesión?
   contexto 61-2026#0  score=0.816
```

```
   contexto 61-2026#1  score=0.695
   contexto 61-2026#3  score=0.653
Respuesta: Gilberth González González es preside la sesión Ordinaria del 2026-10-13.

Pregunta: En el acta 61-2026, ¿qué acta anterior se somete a aprobación?
   contexto 61-2026#0  score=0.770
   contexto 61-2026#1  score=0.697
   contexto 61-2026#3  score=0.675
Respuesta: El acta 61-2026 se somete a aprobación al acta 61-2026.

Pregunta: ¿En qué sesión se acordó enviar una nota de pésame?
   contexto 60-2026#3  score=0.750
   contexto 58-2026#0  score=0.705
   contexto 61-2026#0  score=0.698
Respuesta: La nota de pésame fue enviada en la sesión ordinaria del 2026-10-13.
```

Pregunta	Qué dice el contexto recuperado	¿Correcta?	Alucinación
1. Presidencia	Los hechos del grafo dicen «Presidente: Gilberth González González», y el encabezado `61-2026#0` lo repite.	Sí	Ninguna: la fecha también está en los hechos. Solo un error de redacción («es preside») y no cita el acta.
2. Acta que se aprueba	`61-2026#1`: «somete a aprobación el Acta de la Sesión Ordinaria N°60-2026».	No	Sí: dice que la 61-2026 se aprueba a sí misma. La respuesta (60-2026) estaba en el contexto.
3. Nota de pésame	Ningún fragmento recuperado habla de pésame. `60-2026#3` menciona la «Sesión Extraordinaria No.50-2026»; los otros dos son encabezados.	No (por el contexto)	Sí: afirma una sesión (la ordinaria del 2026-10-13) que el contexto no respalda, y no cita ningún acta.

Pregunta 1. Es la única que funciona como se espera: la pregunta nombra el acta, el filtro deja solo fragmentos de la 61-2026 y el cargo llega como hecho del grafo desde `PARTICIPA_EN`. El modelo solo tiene que copiarlo. La verificación final lo confirma: «la primera respuesta sí nombra a quien el grafo registra como Presidente».

Pregunta 2. La recuperación fue buena: el fragmento `61-2026#1` trae la respuesta textual (la 60-2026). El que falló es el generador: repitió el único id que aparece en la pregunta y en los hechos, y produjo una frase circular. Es una alucinación *dentro* del contexto correcto. La verificación del notebook no la detecta, porque solo revisa que las actas citadas aparezcan en el contexto, y la 61-2026 aparece.

Pregunta 3. Como la pregunta no nombra un acta, `$ids` va vacía y solo cuenta el vector. Los fragmentos que de verdad hablan de las notas de pésame (`61-2026#1` a `#3`, sobre don William Álvarez y don Elías Núñez) no quedaron entre los tres primeros: ganaron un oficio de la 60-2026 y dos encabezados, por el mismo efecto de la pregunta 5. Con ese contexto la respuesta correcta era «no aparece en el contexto». El modelo, en cambio, tomó de los hechos del grafo el tipo y la fecha de la 61-2026 («sesión Ordinaria del 2026-10-13») y los presentó como la sesión de la nota de pésame. Por casualidad coincide con la sesión real, pero el contexto no la respalda: es una alucinación que suena plausible. Tampoco cita un acta, y por eso la verificación da `OK` («ninguna»).

En resumen: cuando la pregunta nombra el acta, el grafo corrige la recuperación (preguntas 1 y 2); cuando no la nombra, queda solo el vector y el contexto puede no contener la respuesta (pregunta 3). Y aun con el contexto correcto, un modelo de 494 millones de parámetros puede equivocarse al leerlo (pregunta 2). Que la verificación pase no significa que la respuesta sea correcta: hay que leerla contra el contexto.